

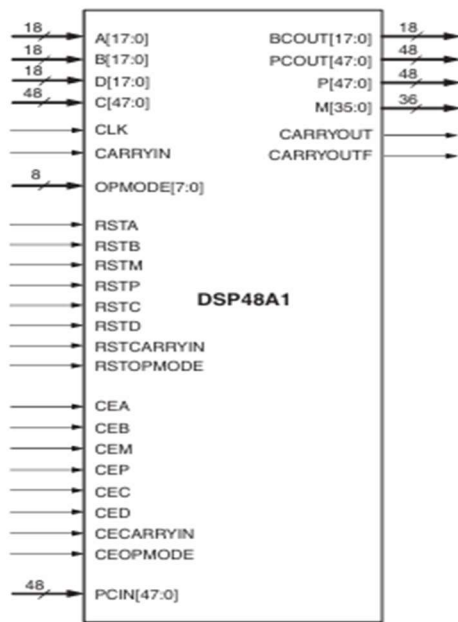
Spartan6-DSP48A1

Prepared by: Aly Khaled

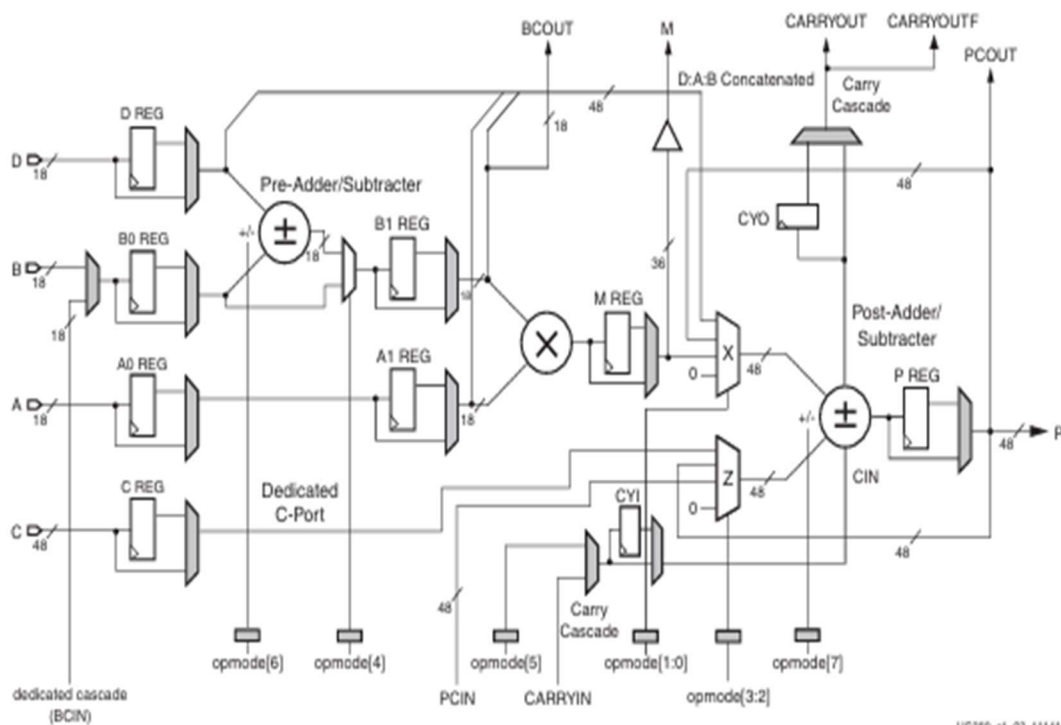
Table of Contents:

DSP48A1 Block.....	3
DSP48A1 Block In Details.....	3
RTL Code.....	4
Reg and mux.....	4
DSP.....	5
Test Bench.....	9
DO File.....	13
Linting.....	13
Questa Sim.....	14
Waveform.....	14
Transcript.....	14
Constraint File.....	15
Elaborating Design.....	15
Synthesis.....	16
Schematic.....	16
Detailed Schematic	16
Zoomed Schematic.....	17
Zoomed Detailed Schematic.....	17
Utilization Report.....	18
Timing Report.....	18
Implementation.....	19
Device.....	19
Utilization Report.....	20
Timing Report.....	20

DSP48A1 Block:



DSP48A1 Block in Detail:

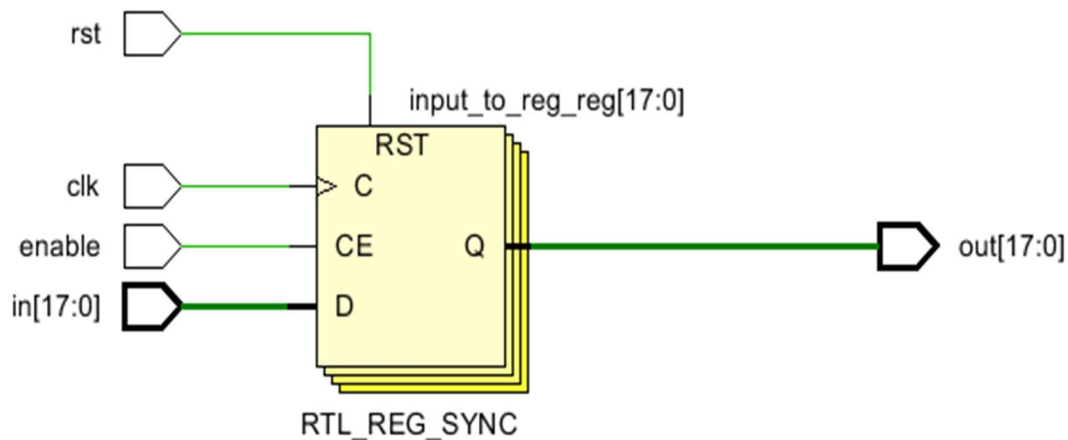


UG289 v1.03 111411

RTL Code:

Reg and mux:

```
1  module reg_and_mux (in,clk,out,rst,enable); //used 10 times
2      parameter rstTYPE="SYNC" ;
3      parameter reg_pipeline=1;
4      parameter width=18; //inputs and outputs width
5      input [width-1:0] in;
6      input clk;
7      input rst;
8      input enable;
9      output [width-1:0] out;
10     reg [width-1:0] input_to_reg;
11     generate
12         if (rstTYPE=="SYNC") begin
13             always @ (posedge clk) begin
14                 if(rst) begin
15                     input_to_reg<=0;
16                 end
17                 else if (enable) begin
18                     input_to_reg<=in;
19                 end
20             end
21         end
22         else if(rstTYPE=="ASYNC") begin
23             always @ (posedge clk or posedge rst) begin
24                 if(rst) begin
25                     input_to_reg<=0;
26                 end
27                 else if (enable) begin
28                     input_to_reg<=in;
29                 end
30             end
31         end
32     endgenerate
33     assign out=(reg_pipeline)?input_to_reg:in;
34 endmodule
35
```



DSP:

```
39 module DSP (A,B,C,D,BCIN,CARRYIN,CLK,OPMODE,CEA,CEB,CEC,CED,CECARRYIN,CEM,CEP,CEOPMODE,  
40 RSTA,RSTB,RSTC,RSTD,RSTCARRYIN,RSTM,RSTP,RSTOPMODE,BCOUT,PCOUT,M,P,PCIN,CARRYOUT,CARRYOUTF);  
41 //parameters  
42 parameter A0REG=0;  
43 parameter A1REG=1;  
44 parameter B0REG=0;  
45 parameter B1REG=1;  
46 parameter CREG=1;  
47 parameter DREG=1;  
48 parameter MREG=1;  
49 parameter PREG=1;  
50 parameter CARRYINREG=1;  
51 parameter CARRYOUTREG=1;  
52 parameter OPMODEREG=1;  
53 parameter CARRYINSEL="OPMODE[5]";  
54 parameter B_INPUT="DIRECT";  
55 parameter RSTTYPE="SYNC";  
56 //input ports  
57 input [17:0] A;  
58 input [17:0] B;  
59 input [47:0] C;  
60 input [17:0] D;  
61 input [17:0] BCIN;  
62 input [47:0] PCIN;  
63 input CARRYIN;  
64 input CLK;  
65 input CEA;  
66 input CEB;  
67 input CEC;  
68 input CECARRYIN;  
69 input CED;  
70 input CEM;  
71 input CEOPMODE;  
72 input CEP;  
73 input RSTA;  
74 input RSTB;  
75 input RSTC;  
76 input RSTCARRYIN;  
77 input RSTD;  
78 input RSTM;  
79 input RSTOPMODE;  
80 input RSTP;  
81 input [7:0] OPMODE;  
82 //output ports  
83 output [35:0] M;  
84 output [47:0] P;
```

```

85 output [47:0] PCOUT;
86 output [17:0] BCOUT;
87 output CARRYOUT;
88 output CARRYOUTF;
89 //internal signals
90 reg [17:0] B_OR_BCIN; //output of the mux before B0 REG
91 wire [17:0] A0_MUX_OUT; //output of A0 REG mux
92 wire [17:0] A1_MUX_OUT; //output of A1 REG mux
93 wire [17:0] B0_MUX_OUT; //output of B0 REG mux
94 wire [17:0] B1_MUX_OUT; //output of B1 REG mux
95 wire [47:0] C_MUX_OUT; //output of C REG mux
96 wire [17:0] D_MUX_OUT; //output of D REG mux
97 wire [7:0] OPMODE_MUX_OUT; //output of OPMODE REG mux
98 wire [35:0] M_MUX_OUT; //output of M REG mux
99 reg [17:0] pre_add_subtract_out; //output of pre-adder/subtractor
100 reg [17:0] pre_B1REG_MUX_OUT; //input of B1 REG
101 reg [35:0] multiplier_out; //output of the multiplier
102 reg CARRY_CASCADE_MUX_OUT; //input of CYI REG
103 wire CYI_MUX_OUT; //output of CYI REG
104 reg [47:0] mux_X_OUT; //MUX X output
105 reg [47:0] mux_Z_OUT; //MUX z output
106 reg [48:0] post_adder_subtract_out; // output of post-adder/subtractor
107 wire [47:0] p_internal;
108 wire carryout_msb; // carryout bit
109 wire [47:0] concatenated_bus;
110 always @ (*) begin //mux 2 to 1 (B and BCIN)
111     if(B_INPUT=="DIRECT") begin
112         B_OR_BCIN=B;
113     end
114     else if (B_INPUT=="CASCADE") begin
115         B_OR_BCIN=BCIN;
116     end
117     else begin
118         B_OR_BCIN=0;
119     end
120 end
121 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(DREG),.width(18)) D_REG (D,CLK,D_MUX_OUT,RSTD,CE);
122 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(B0REG),.width(18)) B0_REG (B_OR_BCIN,CLK,B0_MUX_OUT,RSTB,CEB);
123 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(A0REG),.width(18)) A0_REG (A,CLK,A0_MUX_OUT,RSTA,CEA);
124 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(CREG),.width(48)) C_REG (C,CLK,C_MUX_OUT,RSTC,CEC);
125 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(A1REG),.width(18)) A1_REG (A0_MUX_OUT,CLK,A1_MUX_OUT,RSTA,CEA);
126 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(B1REG),.width(18)) B1_REG (pre_B1REG_MUX_OUT,CLK,B1_MUX_OUT,RSTB,CEB);
127 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(OPMODEREG),.width(8)) OPMODE_REG (OPMODE,CLK,OPMODE_MUX_OUT,RSTOPMODE,CEOPMODE);
128 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(MREG),.width(36)) M_REG (multiplier_out,CLK,M_MUX_OUT,RSTM,CEM);
129 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(CARRYINREG),.width(1)) CYI_REG (CARRY_CASCADE_MUX_OUT,CLK,CYI_MUX_OUT,RSTCARRYIN,CECARRYIN);
130 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(PREG),.width(48)) P_REG (post_adder_subtract_out[47:0],CLK,p_internal,RSTP,CEP);
131 reg_and_mux # (.rstTYPE(RSTTYPE),.reg_pipeline(CARRYOUTREG),.width(1)) CVO_REG (carryout_msb,CLK,CARRYOUT,RSTCARRYIN,CECARRYIN);
132 //post-adder/subtractor operation

```

```

132 //pre-Adder/subtractor operation
133 always @ (*) begin
134     case(OPMODE_MUX_OUT[6])
135         1'b0:pre_add_subtract_out=D_MUX_OUT+B0_MUX_OUT;
136         1'b1:pre_add_subtract_out=D_MUX_OUT-B0_MUX_OUT;
137         default:pre_add_subtract_out=18'b0;
138     endcase
139 end
140 //mux before B1REG
141 always @ (*) begin
142     if(OPMODE_MUX_OUT[4]) begin
143         pre_B1REG_MUX_OUT=pre_add_subtract_out;
144     end
145     else if(OPMODE_MUX_OUT[4]==0) begin
146         pre_B1REG_MUX_OUT=B0_MUX_OUT;
147     end
148 end
149 //multiplier
150 always @ (*) begin
151     multiplier_out=A1_MUX_OUT*B1_MUX_OUT;
152 end
153 //CYI MUX
154 always @ (*) begin
155     if(CARRYINSEL=="CARRYIN") begin
156         CARRY_CASCADE_MUX_OUT=CARRYIN;
157     end
158     else if(CARRYINSEL=="OPMODE[5]") begin
159         CARRY_CASCADE_MUX_OUT=OPMODE_MUX_OUT[5];
160     end
161 end

```

```

165 //mux X
166 always @(*) begin
167     case (OPMODE_MUX_OUT[1:0])
168         0: mux_X_OUT=0;
169         1: mux_X_OUT={12'b0,M_MUX_OUT};
170         2: mux_X_OUT=p_internal;
171         3: mux_X_OUT=concatenated_bus;
172     endcase
173 end
174 //mux Z
175 always @(*) begin
176     case (OPMODE_MUX_OUT[3:2])
177         0: mux_Z_OUT=0;
178         1: mux_Z_OUT=PCIN;
179         2: mux_Z_OUT=p_internal;
180         3: mux_Z_OUT=C_MUX_OUT;
181     endcase
182 end
183
184 //post-adder/subtractor
185 always @ (*) begin
186     if(OPMODE_MUX_OUT[7]) begin
187         post_adder_subtract_out=(mux_Z_OUT-(mux_X_OUT+CYI_MUX_OUT));
188     end
189     else if (OPMODE_MUX_OUT[7]==0) begin
190         post_adder_subtract_out=mux_X_OUT+mux_Z_OUT+CYI_MUX_OUT;
191     end
192     else begin
193         post_adder_subtract_out=48'b0;
194     end
195 end
196 //outputs
197 assign P=p_internal;
198 assign PCOUT=p_internal;
199 assign carryout_msb=post_adder_subtract_out[48];
200 assign CARRYOUTF=CARRYOUT;
201 assign BCOUT=B1_MUX_OUT;
202 assign M=M_MUX_OUT;
203 endmodule

```

Test Bench:

```
1  module DSP_tb();
2  //Parameters
3  parameter A0REG=0;
4  parameter A1REG=1;
5  parameter B0REG=0;
6  parameter B1REG=1;
7  parameter CREG=1;
8  parameter DREG=1;
9  parameter MREG=1;
10 parameter PREG=1;
11 parameter CARRYINREG=1;
12 parameter CARRYOUTREG=1;
13 parameter OPMODEREG=1;
14 parameter CARRYINSEL="OPMODE[5]";
15 parameter B_INPUT="DIRECT";
16 parameter RSTTYPE="SYNC";
17 //Inputs
18 reg [17:0] A;
19 reg [17:0] B;
20 reg [47:0] C;
21 reg [17:0] D;
22 reg [17:0] BCIN;
23 reg [47:0] PCIN;
24 reg CARRYIN;
25 reg CLK;
26 reg CEA;
27 reg CEB;
28 reg CEC;
29 reg CECARRYIN;
30 reg CED;
31 reg CEM;
32 reg CEOPMODE;
33 reg CEP;
34 reg RSTA;
35 reg RSTB;
36 reg RSTC;
37 reg RSTCARRYIN;
38 reg RSTD;
39 reg RSTM;
40 reg RSTOPMODE;
41 reg RSTP;
42 reg [7:0] OPMODE;
43 //Outputs
44 wire [35:0] M;
45 wire [47:0] P;
46 wire [47:0] PCOUT;
47 wire [17:0] BCOUT;
48 wire CARRYOUT;
49 wire CARRYOUTF;
50 //SIGNALS
51 reg [47:0] preP;
52 reg [47:0] prePOUT;
53 //OUT
54 DSP #(A0REG(A0REG), A1REG(A1REG), B0REG(B0REG), B1REG(B1REG), CREG(CREG), DREG(DREG),
55 MREG(MREG), PREG(PREG), CARRYINREG(CARRYINREG), CARRYOUTREG(CARRYOUTREG), OPMODEREG(OPMODEREG),
56 CARRYINSEL(CARRYINSEL), B_INPUT(B_INPUT), RSTTYPE(RSTTYPE))
57 dsp_dut (A,B,C,D,BCIN,CARRYIN,CLK,OPMODE,CEA,CEB,CEC,CED,CECARRYIN,CEM,CEP,CEOPMODE,
58 RSTA,RSTB,RSTC,RSTD,RSTCARRYIN,RSTM,RSTP,RSTOPMODE,BCOUT,PCOUT,M,P,PCIN,CARRYOUT,CARRYOUTF);
```

```

59 //Clock generation
60 initial begin
61     CLK=0;
62     forever begin
63         #1;
64         CLK=~CLK;
65     end
66 end
67 initial begin
68     RSTA=1;
69     RSTB=1;
70     RSTC=1;
71     RSTD=1;
72     RSTCARRYIN=1;
73     RSTM=1;
74     RSTOPMODE=1;
75     RSTP=1;
76     A=$random;
77     B=$random;
78     C=$random;
79     D=$random;
80     BCIN=$random;
81     CARRYIN=$random;
82     OPMODE=$random;
83     CEA=$random;
84     CEB=$random;
85     CEC=$random;
86     CED=$random;
87     CEM=$random;
88     CECARRYIN=$random;
89     CEOPMODE=$random;
90     CEP=$random;
91     @ (negedge CLK);
92     if (M==0 && P==0 && PCOUT==0 && BCOUT==0 && CARRYOUT==0 && CARRYOUTF==0 ) begin
93         $display("CORRECT OUTPUT-The reset is working well");
94     end
95     else begin
96         $display("ERROR-INCORRECT OUTPUT");
97         $stop;
98     end

```

```

99     RSTA=0;
100    RSTB=0;
101    RSTC=0;
102    RSTD=0;
103    RSTCARRYIN=0;
104    RSTM=0;
105    RSTOPMODE=0;
106    RSTP=0;
107    CEA=1;
108    CEB=1;
109    CEC=1;
110    CED=1;
111    CEM=1;
112    CECARRYIN=1;
113    CEOPMODE=1;
114    CEP=1;
115    repeat(100) begin
116        OPMODE=S'b11011101;
117        A=20;
118        B=10;
119        C=350;
120        D=25;
121        BCIN=$random;
122        PCIN=$random;
123        CARRYIN=$random;
124        repeat(4) @ (negedge CLK);
125        if(BCOUT==hf && M==h12c && P==h32 && PCOUT==h32 && CARRYOUT==0 && CARRYOUTF==0 ) begin
126            $display("CORRECT OUTPUT-path 1 works well");
127        end
128        else begin
129            $display("ERROR-INCORRECT OUTPUT");
130            $stop;
131        end
132    end
133
134    repeat(100) begin`
135        OPMODE = S'b00010000;
136        A = 20;
137        B = 10;
138        C = 350;
139        D = 25;
140        BCIN=$random;
141        PCIN=$random;
142        CARRYIN=$random;
143        repeat(3) @(negedge CLK);
144        if(BCOUT==h23 && M==h2bc && P==0 && PCOUT==0 && CARRYOUT==0 && CARRYOUTF==0 ) begin
145            $display("CORRECT OUTPUT-path 2 works well");
146        end
147        else begin
148            $display("ERROR-INCORRECT OUTPUT");
149            $stop;
150        end
151    end

```

```

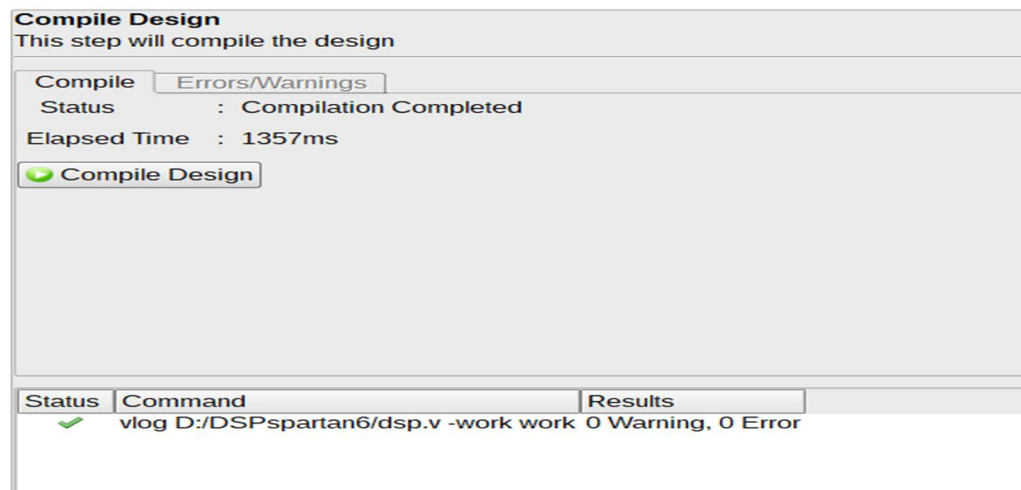
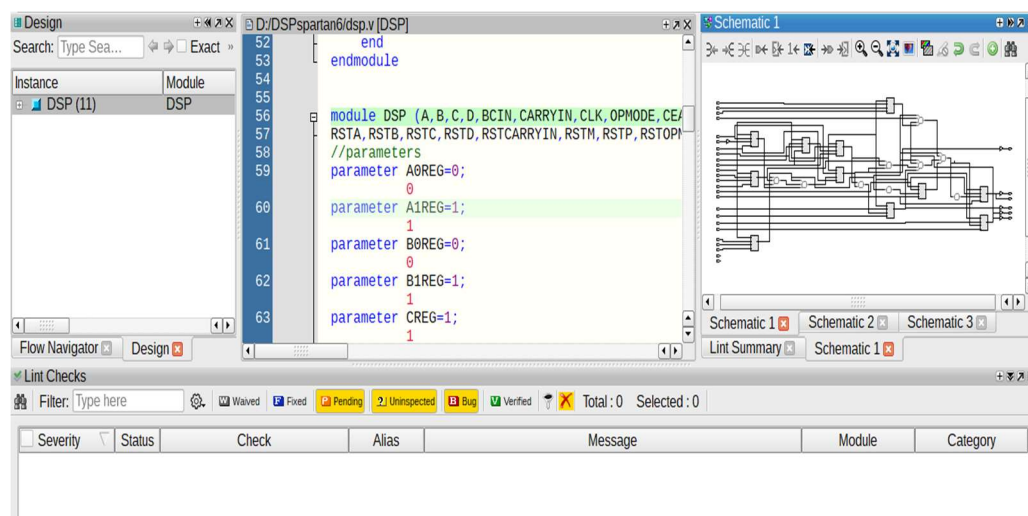
153     repeat (100) begin
154         OPMODE = 8'b00001010;
155         A = 20;
156         B = 10;
157         C = 350;
158         D = 25;
159         BCIN=$random;
160         PCIN=$random;
161         CARRYIN=$random;
162         preP=P;
163         prePOUT=PCOUT;
164         repeat(3) @(negedge CLK);
165         if(BCOUT=='ha && M=='hc8 && P==preP && PCOUT==prePOUT && CARRYOUT==0 && CARRYOUTF==0 ) begin
166             $display("CORRECT OUTPUT-path 3 works well");
167         end
168         else begin
169             $display("ERROR-INCORRECT OUTPUT");
170             $stop;
171         end
172     end
173     repeat(100) begin
174         OPMODE = 8'b10100111;
175         A = 5;
176         B = 6;
177         C = 350;
178         D = 25;
179         PCIN=3000;
180         BCIN=$random;
181         CARRYIN=$random;
182         repeat(3) @(negedge CLK);
183         if(BCOUT=='h6 && M=='h1e && P=='hfe6fffec0bb1 && PCOUT=='hfe6fffec0bb1 && CARRYOUT==1 && CARRYOUTF==1 ) begin
184             $display("CORRECT OUTPUT-path 4 works well");
185         end
186         else begin
187             $display("ERROR-INCORRECT OUTPUT");
188             $stop;
189         end
190     end
191 end
192 end
193 endmodule

```

DO File:

```
DSPspartan6 > run.do
1  vlib work
2  vlog dsp.v DSPtestbench.v
3  vsim -voptargs=+acc work.DSP_tb
4  add wave *
5  run -all
```

Linting:



Waveform:

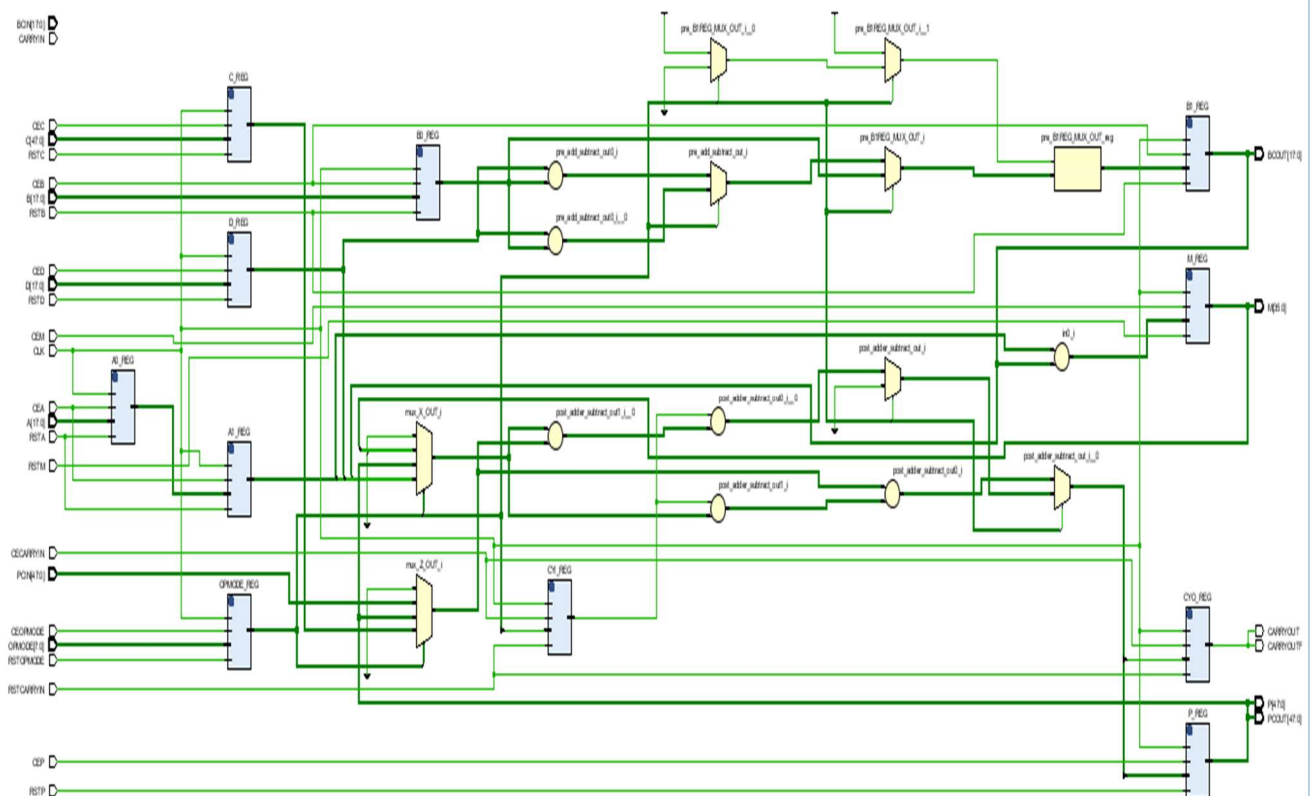


Page | 14

Constraint File:

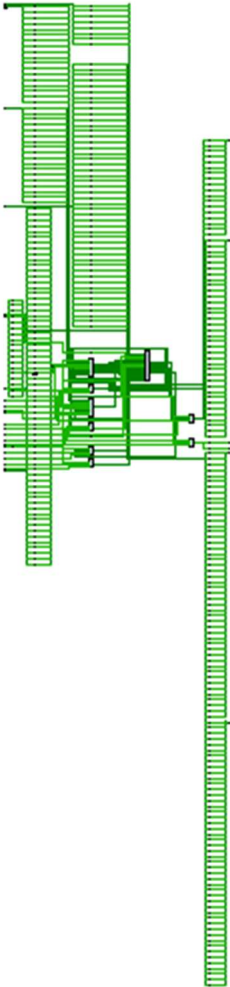
```
DSPspartan6 > @ Constraints_basys3.xdc
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  ## Clock signal
7  set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports CLK]
8  create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports CLK]
```

Elaborated Design:

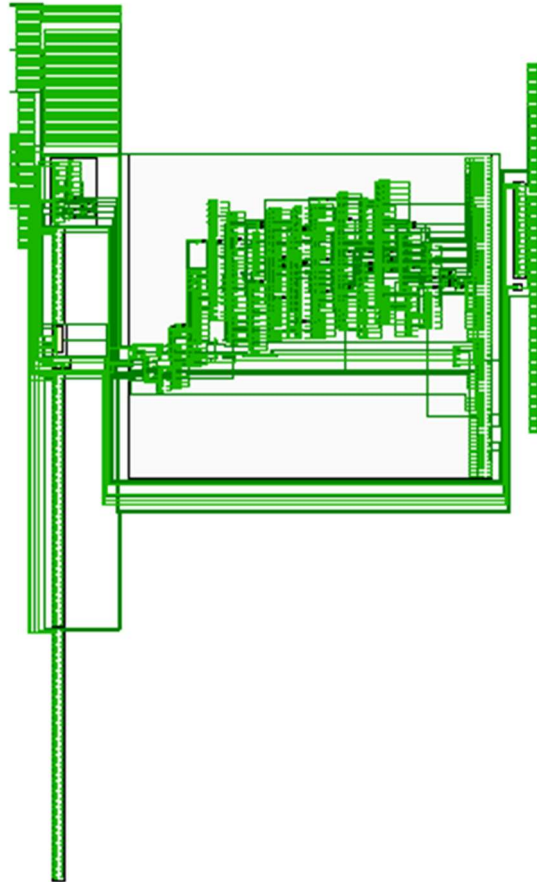


Synthesis:

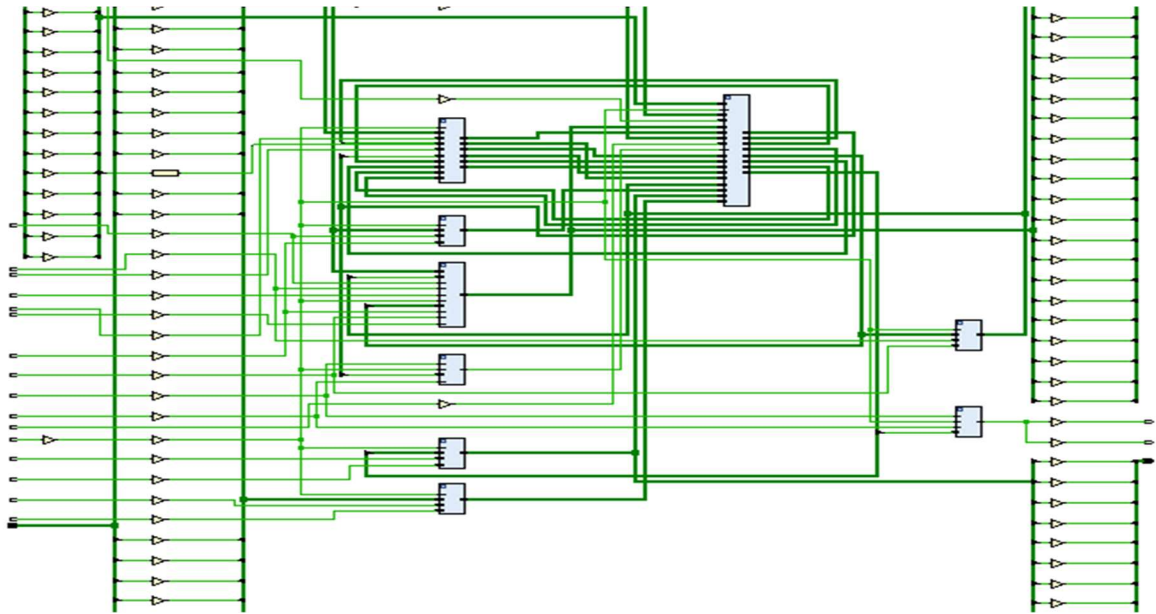
Schematic:



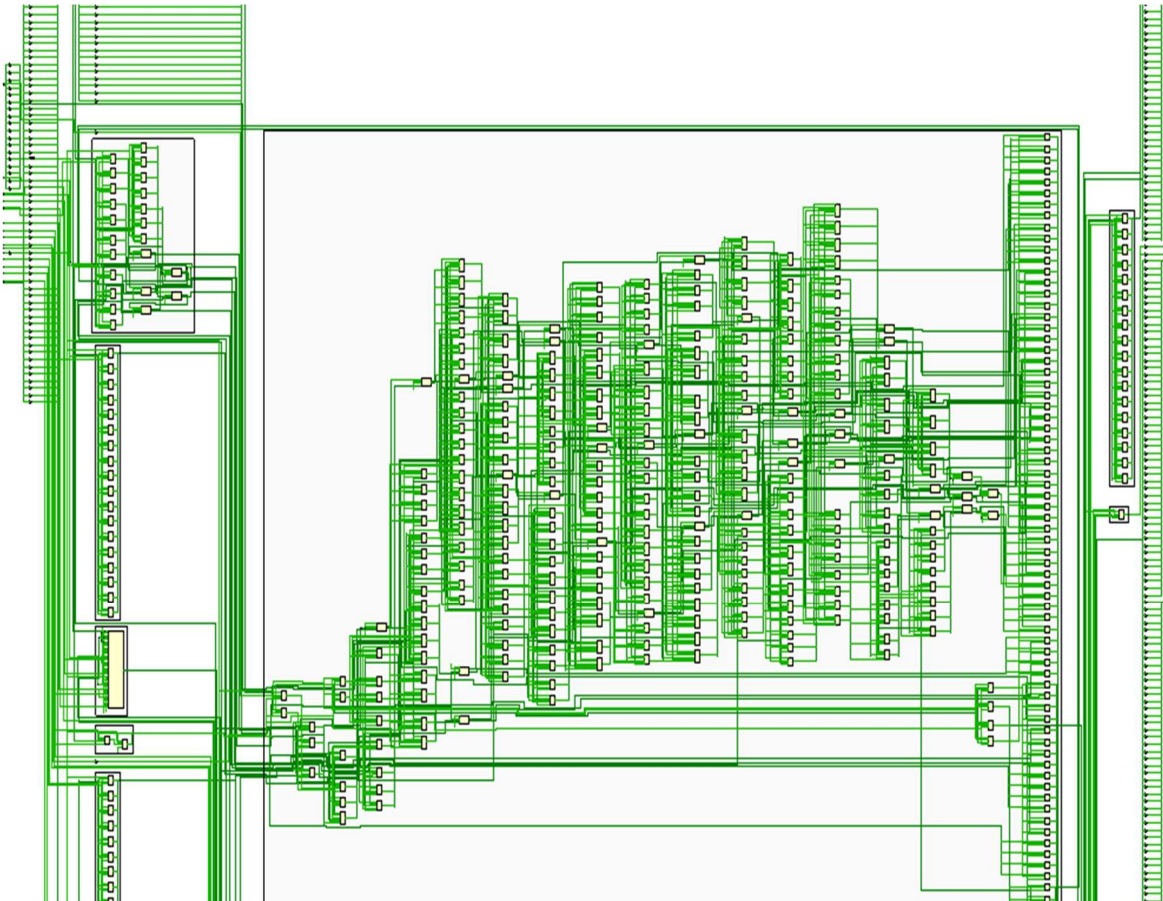
Detailed Schematic:



Zoomed Schematic:



Detailed Zoomed Schematic:



Utilization Report:

Name	Slice LUTs (134600)	Slice Registers (269200)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
▼ DSP	289	160	1	327	1
A1_REG (reg_and_mux)	0	18	0	0	0
B1_REG (reg_and_mu...	0	18	0	0	0
C_REG (reg_and_mux...	0	48	0	0	0
CYI_REG (reg_and_m...	1	1	0	0	0
CYO_REG (reg_and_...	0	1	0	0	0
D_REG (reg_and_mux...	0	18	0	0	0
dbg_hub (dbg_hub_CV)	0	0	0	0	0
M_REG (reg_and_mux...	0	0	1	0	0
OPMODE_REG (reg_a...	287	8	0	0	0
P_REG (reg_and_mux...	0	48	0	0	0
u_ila_0 (u_ila_0_CV)	0	0	0	0	0

Timing Report:

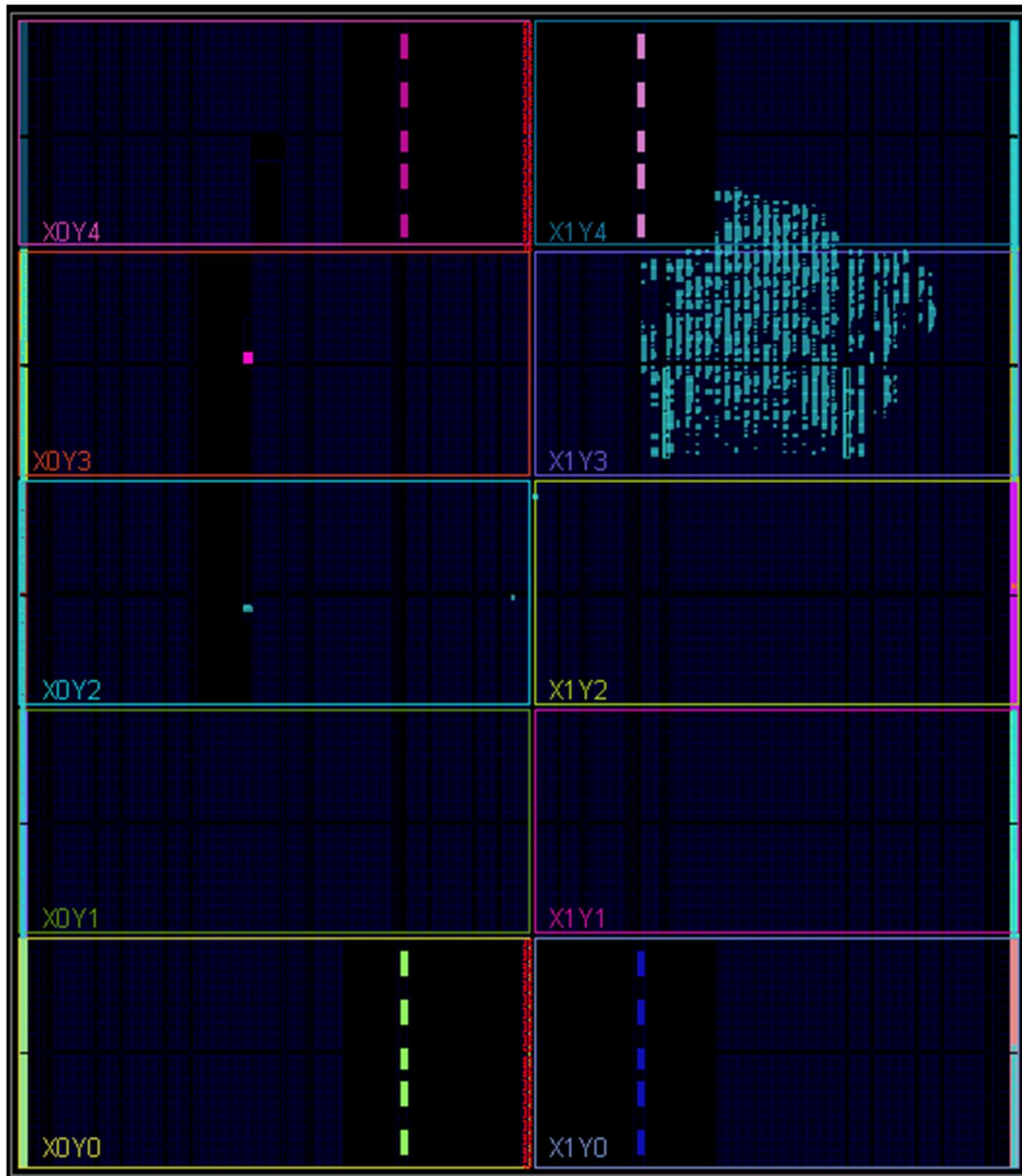
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.224 ns	Worst Hold Slack (WHS): 0.182 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 87	Total Number of Endpoints: 87	Total Number of Endpoints: 162

All user specified timing constraints are met.

Implementation:

Device:



Implementation Utilization Report:

Name	Slice LUTs (133800)	Slice Registers (267600)	F7 Muxes (66900)	F8 Muxes (33450)	Slice (33450)	LUT as Logic (133800)	LUT as Memory (46200)	LUT Flip Flop Pairs (133800)	Block RAM Tile (365)	DSP s (740)	Bonded IOB (500)	BUFCTRL (32)	BSCANE2 (4)
▼ DSP	2762	4225	95	12	1602	2288	474	1544	8	1	327	2	1
A1_REG (reg_and_mux)	0	18	0	0	6	0	0	0	0	0	0	0	0
B1_REG (reg_and_mu...	0	18	0	0	8	0	0	0	0	0	0	0	0
C_REG (reg_and_mux...	0	48	0	0	19	0	0	0	0	0	0	0	0
CY1_REG (reg_and_m...	1	1	0	0	1	1	0	1	0	0	0	0	0
CY0_REG (reg_and_...	0	1	0	0	1	0	0	0	0	0	0	0	0
D_REG (reg_and_mux...	0	18	0	0	13	0	0	0	0	0	0	0	0
> dbg_hub (dbg_hub)	476	727	0	0	268	452	24	305	0	0	0	1	1
M_REG (reg_and_mux...	0	0	0	0	0	0	0	0	0	1	0	0	0
OPMODE_REG (reg_a...	287	8	0	0	87	287	0	0	0	0	0	0	0
P_REG (reg_and_mux...	0	48	0	0	14	0	0	0	0	0	0	0	0
> u_ila_0 (u_ila_0)	1997	3338	95	12	1229	1547	450	1207	8	0	0	0	0

Implementation Timing Report:

Design Timing Summary											
Setup				Hold				Pulse Width			
Worst Negative Slack (WNS): 2.580 ns				Worst Hold Slack (WHS): 0.059 ns				Worst Pulse Width Slack (WPWS): 3.950 ns			
Total Negative Slack (TNS): 0.000 ns				Total Hold Slack (THS): 0.000 ns				Total Pulse Width Negative Slack (TPWS): 0.000 ns			
Number of Failing Endpoints: 0				Number of Failing Endpoints: 0				Number of Failing Endpoints: 0			
Total Number of Endpoints: 8058				Total Number of Endpoints: 8042				Total Number of Endpoints: 5136			
All user specified timing constraints are met.											